



Master Thesis

Swarm Drones Control Platform

A modular and scalable platform for swarm drones control

Nicolas Leboucher

November 17, 2024

Supervisors

Dr Xiao Xiao and Dr Marc Teyssier Professors at the Institute
for Future Technologies

Contents

Contents	1
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Research Objectives	2
1.4 Significance of the Research	3
1.5 Scope and Limitations	3
1.6 Structure Overview	4
2 State of the Art	4
2.1 Safe Drone Flight Around Humans	4
2.2 Controlling Drones with Human Input	5
2.3 Technological Platforms for Swarm Drones	5
2.4 Human Pose Detection for Drone Control	6
2.5 Gaps in Human-Swarm Interaction Platforms	7
3 Contribution 1: Swarm Control	8
3.1 Introduction	8
3.2 Drone Control Kernel (DCK a drone localization and control system)	8
3.3 Evaluation and Results	12
3.4 Conclusion	12
4 Contribution 2: Human Pose Estimation System	13
4.1 Introduction	13
4.2 Evaluation and Results	16
4.3 Conclusion	16
5 Contribution 3: Platform : Human-Swarm Interaction	18
5.1 Introduction	18
5.2 Platform for Human-Swarm Interaction	18
5.3 Opportunitites	19
6 Conclusion and Opportunities for Future Work	21
References	24
Special Terms	26

1 Introduction

1.1 Background and Motivation

Swarm robotics is an emerging field in robotics inspired by biological systems, such as flocks of birds or colonies of ants, where simple units collaborate to achieve complex tasks. In recent years, advancements in drone technology have expanded the potential applications of swarm robotics, including environmental monitoring, search-and-rescue operations, and industrial automation [1]. In particular, drones operating as swarms offer a promising solution for tasks requiring adaptability and scalability.

One key area of interest is the integration of drone swarms into environments shared with humans. Applications such as drone-based assistance systems in healthcare, smart warehouses, and collaborative robotics emphasize the need for reliable, safe human-drone interaction [1]. The use of drones in indoor environments, where traditional positioning systems such as GPS are not viable, presents both opportunities and challenges, particularly when human safety and interaction are prioritized.

In this context, existing swarm control platforms like Crazyswarm and CFlib offer robust tools for drone coordination and control. However, these platforms often require specialized equipment, such as motion capture systems, and rely on Python-based libraries, limiting accessibility for non-expert users and those working in non-research settings [2, 3]. This raises a gap: a need for more accessible, intuitive control systems that allow seamless interaction between human operators and drone swarms in real-time indoor environments.

1.2 Problem Statement

While existing platforms provide powerful tools for swarm control, they do not address the challenge of creating a system that integrates human pose detection with real-time drone control in a user-friendly and accessible way. Most current systems are limited in their ability to allow developers to work directly with drones in environments where they need to safely interact with human operators.

This research addresses this gap by proposing a web-based platform for controlling swarms of Crazyflie drones in indoor spaces using real-time human pose estimation. The platform aims to enable intuitive and responsive interaction between humans and drones without requiring complex external systems.

In figure 1a we present a scene of drones and all detection systems controlled by humans; figure 1b concerns the web control and monitoring. Similarly in red is the drone system and in yellow the human system with the human pose detection. In red above there is the drone localization system. In grey above is the human localization system.

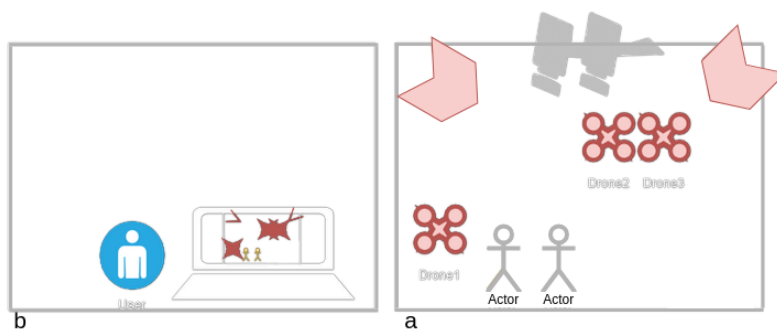


Figure 1: (a) Inside the drone room. Similarly in red is the drone system and in grey the human system with the human localization kernel. In red above there is the drone localization and control system. In grey above is the human localization system. (b) Outside the room, a user is controlling and/or monitoring the drones. He is able to see the drones and humans in the 3D space using the web interface.

1.3 Research Objectives

This research aims to address the limitations of existing swarm control platforms by developing a user-friendly, web-based system for controlling swarms of drones in indoor environments where interaction with

humans is required. More precisely the objectives of this research are as follows:

- ▶ Develop a web-based platform for controlling swarms of Crazyflie drones, enabling easier access for developers, researchers, and non-experts.
- ▶ Integrate real-time human pose detection using affordable and accessible technologies, such as Intel RealSense and MediaPipe [4].
- ▶ Ensure safe and responsive drone interactions in indoor environments by integrating collision avoidance systems and real-time feedback.
- ▶ Evaluate the platform's performance in controlled and dynamic indoor environments with human interaction.

This research will demonstrate how a **web-based interface**, coupled with **advanced human pose detection**, can make drone swarm control more accessible and enhance human-drone collaboration in shared spaces.

1.4 Significance of the Research

The proposed platform has the potential to significantly impact the field of swarm robotics by broadening access to swarm control technologies. Currently, systems like *CrazySwarm* and *CFLib* require Python-based development, which limits their use to researchers or developers with specific technical expertise [2, 3]. By shifting to a **web-based architecture**, this research will make drone control systems more accessible to a wider audience, enabling cross-platform development and more intuitive interfaces.

Furthermore, by integrating real-time human pose detection, this platform introduces a **novel method for enabling drones to interact with humans safely in indoor environments**. This has wide-ranging applications, from smart warehouses, where drones assist human workers [5, 6]. **The platform also has the potential to be used in educational settings**, allowing students to interact with drones without complex technical setups.

Ultimately, this research fills a critical **gap** in current swarm robotics systems, creating a foundation for future development in both research and practical applications, particularly in areas **where drone-human interaction is necessary**.

1.5 Scope and Limitations

The proposed platform has limitations that should be considered. First, the platform is designed for indoor environments, where GPS signals are unreliable, and human interaction is a primary concern. It can not work outdoors.

The platform will not address internal positioning of the drones, only **external positioning**.

The platform will not address drone swarm control in outdoor or GPS-based environments.

The platform will use real-time human pose detection technologies like Intel RealSense and MediaPipe, which may have limitations in complex environments.

Despite these limitations, the proposed platform offers a significant advancement in swarm robotics by providing a user-friendly interface for controlling drones in indoor environments around humans. The platform's focus on accessibility and safety makes it a valuable tool for researchers, developers, and educators interested in swarm robotics and human-drone interaction.

1.6 Structure Overview

In the following chapters, we will present the design, implementation, and evaluation of the proposed platform for controlling drone swarms in indoor environments with human interaction. Chapter 2 Reviews the State of the Art, including existing swarm control platforms like CrazySwarm, ModQuad, and human pose detection systems.

Chapter 3 Describes the design and implementation of the proposed Drone Control Kernel.

Chapter 4 Describes the integration of real-time human pose detection.

Chapter 5 Presents the platform for human-swarm interaction will present the experimental setup and results, validating the platform's functionality including possible extensions of the platform for larger-scale deployment.

Chapter 6 Discusses the reliability and quality of the platform, and the improvements with respect to the previous systems. Also the platform functionalities are suggested for the near future.

2 State of the Art

In recent years, swarm drone technology has advanced significantly, leading to breakthroughs in human-drone interaction and multi-robot systems. While several research efforts have focused on the safety, control, and interaction between drones and humans, there remains a gap in the availability of robust platforms for developers to control drone swarms indoors around humans. This section will explore existing works that demonstrate safe operation of drones around humans, systems that enable real-time drone control based on human input, and the identified gaps in the current technological landscape.

2.1 Safe Drone Flight Around Humans

Safety is a primary concern in close proximity, especially in confined spaces such as indoor environments. Several research efforts presented in the following are focused on ensuring that drones can operate around humans without posing significant risks [7].

One such project is Drone & Me, which investigates how drones can safely navigate around humans by recognizing human poses and responding

accordingly [7]. In addition, Ju et al. (2017) proposed real-time trajectory adjustments for drones to ensure safe flight paths when operating around humans indoors. The last systems emphasize the importance of autonomous collision avoidance and the use of protective hardware to mitigate risks [8].

2.2 Controlling Drones with Human Input

Beyond safety, there has been significant progress in enabling human control over drones through various input methods, such as body gestures, hand movements, or wearable technology. The FlyJacket platform, for instance, allows a user to intuitively control a drone by wearing an exoskeleton that translates upper body movements into flight commands [5]. This system demonstrates that drones can be controlled naturally and in real-time based on human gestures, but it is limited to the control of a single drone.

In a more general approach, Cai et al. (2019) explored how drones can follow human postures and gestures using computer vision, allowing more flexible interactions with drones [9]. These systems highlight the potential for intuitive drone control but lack the scalability needed to control swarms of drones effectively.

2.3 Technological Platforms for Swarm Drones

Several robust platforms have been developed to enable scalable swarm drone systems. Among these, Crazyflie is a widely used platform for research on nano-quadrotor swarms. It provides the infrastructure for advanced coordination and trajectory planning of multiple drones in a shared airspace. The platform's compatibility with tools like ROS (Robot Operating System) makes it a flexible tool for swarm applications [2].

Another well-known platform is Kilobot, developed by Harvard's Wyss Institute. Though primarily a ground-based swarm robotics platform, Kilobot's decentralized control algorithms provide valuable insights into swarm intelligence and distributed behaviors, which are applicable to aerial drones as well [10]. ModQuad further builds on swarm modularity by allowing multiple drones to self-assemble mid-air, forming complex structures that can adapt to various tasks [11].

The CrazyChoir platform demonstrates how multiple Crazyflie drones can perform synchronized movements in an artistic or musical context, emphasizing scalability in both control and safety [12]. These platforms provide strong technological foundations for swarm robotics, yet none fully integrate human interaction into swarm control in indoor environments.

CrazySwarm [13] represents the current state of the art for large-scale swarm control in research environments, particularly in fields like swarm robotics, distributed systems, and human-swarm interaction. It leverages ROS for modularity and scalability, and its integration with real-time motion capture systems provides a sophisticated platform for studying collective behaviors. Conversely, CFLib Swarm focuses on providing

accessible, low-level control of Crazyflie drones for smaller-scale applications, mainly for educational purposes and basic experimentation.

CrazySwarm provides a more advanced, scalable solution for handling large swarms in complex environments. It overcomes the limitations of CFLib [3] Swarm, such as lack of real-time positioning and high-level control, making it the preferred platform for research in swarm robotics.

We want to note here that in the Institute for Future Technologies there was a project that started with ROS but had issues as ROS adds a layer of abstraction and infrastructure that provides many advantages but also introduces overhead. [14]. CFLib avoids this overhead by interacting directly with the drones in a more lightweight package.

2.4 Human Pose Detection for Drone Control

Human-drone interaction has advanced with technologies that allow drones to react to human movements. One notable platform is FlyJacket, a wearable exoskeleton that translates upper body movements into drone control. FlyJacket emphasizes intuitive and immersive control, but it is focused on single-drone systems [5] and figure 2.

Beyond specialized wearables, recent advances in computer vision have made it possible to detect human poses and gestures without specialized hardware. Intel RealSense, combined with MediaPipe, allows real-time human pose estimation from RGB and depth cameras. This technology can detect skeletal movements and translate them into commands for drone control [4] and figure 3. The system's accuracy and low latency make it highly applicable to environments where humans and drones must coexist.

In addition, Cai et al. (2019) proposed a monocular vision system for real-time posture recognition, where drones respond dynamically to human gestures in indoor environments. This system allows for direct interaction between humans and drones but does not yet support multi-drone systems [9].

Leap Motion offers another avenue for gesture-based control by detecting hand and finger movements. While Leap Motion has been successfully integrated with single drones, there is still potential to expand this to swarm systems for more complex indoor tasks [17].

The paper titled "Drone control using Kinect-based posture detection" [4] presents a compelling approach to enhancing human-drone interaction by leveraging RGB-D *Red, Green, Blue – Depth* data from the Kinect sensor. The authors demonstrate how Kinect can effectively capture real-time human poses and positions, enabling drones to respond to user gestures with a high degree of accuracy. By utilizing the depth data, the system can determine the spatial relationship between the human operator and the drone, allowing for intuitive control commands based on body movements. This approach significantly reduces the need for additional hardware or complex controllers [16], as it translates natural human actions directly into drone maneuvers. The paper emphasizes that such a system can enhance the usability and accessibility of drones, particularly in indoor environments where GPS signals may be unreliable.

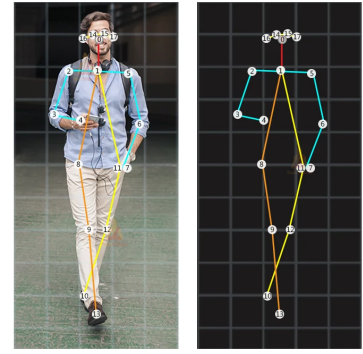


Figure 2: Using mediapipe we can also get the skeleton of the person.



Figure 3: FPV Drone Control using the flyjacket.[15]

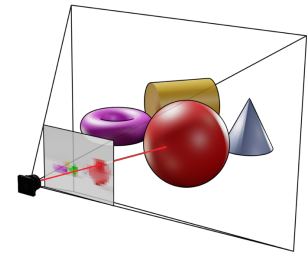


Figure 4: a[16]

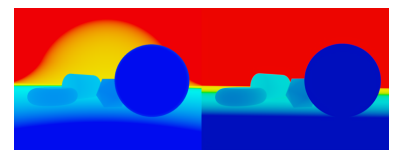


Figure 5: b[16] *a* Projection to camera coordinates *b* Distance map *left* vs Depth map *right* The two images show the difference between a distance map and a depth map. The distance map is a 2D representation of the distance between the camera and the object, while the depth map is a 3D representation of the same distance between the camera and the object.



Figure 6: We can combine the depth map with the RGB image to get a skeleton of the person.

Furthermore, the authors highlight the potential for combining this technology with other methods, such as those explored in [4] and [5], to create a more robust human-drone interaction framework that addresses current limitations in the field.

To summarize, two main approaches are used to detect human poses: depth-based and RGB-based. Depth-based methods, such as Kinect, use depth sensors to capture the 3D structure of the human body. These methods are not robust to lighting conditions but provide accurate depth information. RGB-based methods, such as MediaPipe, use RGB cameras to detect human poses. These methods are more robust to lighting conditions but may lack depth information. By combining depth and RGB data, researchers can create more accurate and robust human pose detection systems. Thus using mediapipe to detect human poses and then using the depth data to get the distance between the drone and the human operator.

2.5 Gaps in Human-Swarm Interaction Platforms

Despite advancements both in safety mechanisms and control systems, there remains a significant gap in providing a unified platform that enables developers to control swarms of drones indoors with same human interactions. Current platforms like Crazyflie and ModQuad provide scalable solutions for drone swarms but lack the necessary integration with human pose recognition and safety protocols for operating indoors around people [2, 11]. Furthermore, while platforms like FlyJacket enable human control, they focus on single-drone control, leaving a gap in swarming capabilities.

Thus, a key challenge remains in the development of a platform that can:

Enable real-time control of multiple drones based on human input (e.g., gestures, body movements). That includes real-time posture and position as all platforms do not seem to include. [3, 5, 18] Ensure safe flight paths in indoor environments.

Make accessible to developers for customization and open-source integration. Python is accessible but we want to be able to use these drones from more platforms or a web service to not require client-drone connection.

3 Contribution 1: Swarm Control

3.1 Introduction

In this section, we present our first contribution, which is the improvement of the Drone Control Kernel. We present the software architecture that we have developed to control drones in a swarm. We also present the results of the evaluation of the DCK.

We were provided a synchronous control driven API we have worked around it to handle practical **asynchronous** calls. We show here this work. Then we have tested it with the drones and have seen that the drones were able to follow a path with a high degree of accuracy. The drones were able to maintain their position and orientation while moving along the path, demonstrating the effectiveness of the control system.

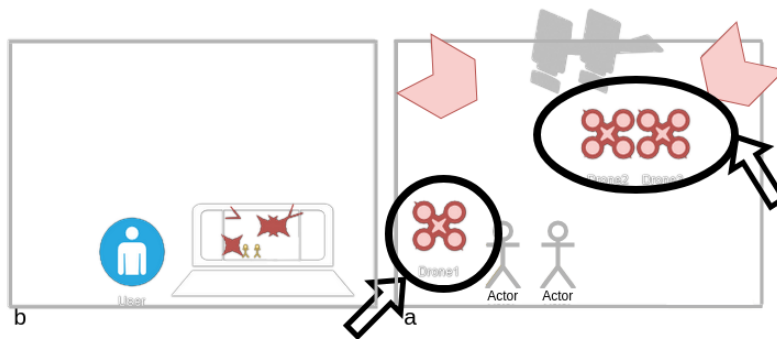


Figure 7: (a) Inside the drone room. Similarly in red is the drone system and in grey the human system with the human localization kernel. In red above there is the drone localization and control system. In grey above is the human localization system. Here we focus on the Drone Control Kernel (DCK). (b) Outside the room, a user is controlling and or monitoring the drones. He is able to see the drones and humans in the 3D space using the web interface

3.2 Drone Control Kernel (DCK a drone localization and control system)

Introduction, Motivation and Objectives

Currently, Crazyflie drones are controlled using their Python library, `cflib`, which provides developers with a robust toolkit for sending commands, receiving sensor data, and managing flight tasks. While this library offers powerful capabilities, it limits the flexibility of using the Python ecosystem, as direct access to the drones requires interacting through this specific interface. As a result, broader accessibility for developers or users unfamiliar with Python is restricted, making it challenging for people from diverse technical backgrounds to engage with drone control.

To address this limitation and open drone control to a wider audience, we propose a **web-based control platform**. By developing a web interface, we aim to abstract the complexities of the underlying Python-based system and make the control of drone swarms more accessible to a larger range of developers, including those familiar with other technologies. This approach democratizes drone control, allowing users to interact with Crazyflie drones through web-based APIs and interfaces, facilitating easier integration into various applications, including human-drone interactions in indoor environments.

Instantaneous API Calls and Background Processing: Asynchronous APIs enhance the functionality of drone control applications by handling requests in the background. This approach maintains application functionality while keeping resources free to process new requests. Especially

in drone operations where real-time responsiveness is crucial, this capability ensures efficient handling of multiple tasks simultaneously.

Adaptability to Connectivity and Execution Time: Asynchronous APIs are particularly beneficial in environments with variable connectivity or where requests have longer execution times. They allow for the processing of complex operations without causing delays in the application's responsiveness, crucial for real-time drone control where delay can have significant consequences.

Event-Driven Communication: Asynchronous APIs, enable more intelligent communication between internal and external services. This is especially important in drone operations where multiple events or commands may need to be handled in real time, ensuring smoother and more efficient workflows

Support for Various Protocols: These APIs support a range of messaging protocols and transports such as WebSockets and UDP subscriptions, which are essential for the dynamic and varied requirements of drone control. This flexibility allows for fast and more accessible and versatile drone operation management.

Heritage and Components Used in the Project

The DCK consists of several components that work together to provide a robust and flexible control system for managing drone swarms. Crazyflie drones being used in this project communicate using the **Radio component Crazy Radio PA (CRPA)**. We include also decks that can be added to the drones to add the location of the drones using **Lighthouse** technology. This requires a specific deck to be added to the drones and Lighthouse or bases to be placed around the room. The drones are then able to locate themselves in the room. [14]

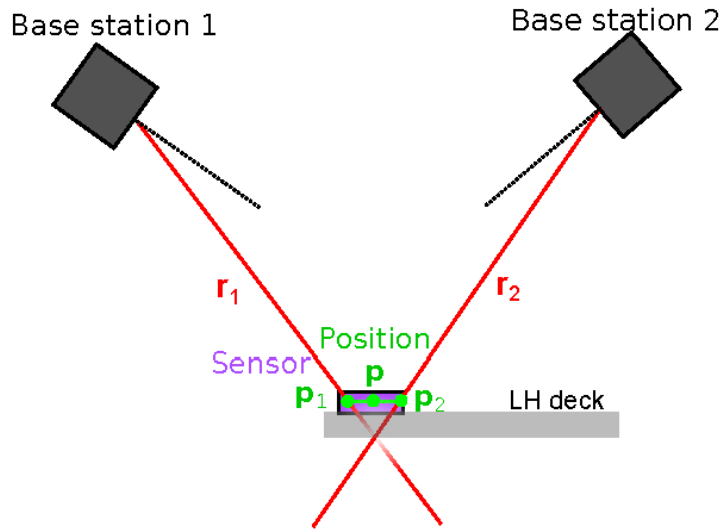
Theory

According to the state of the art the drones can be controlled using different localization methods. The lab's legacy left us the Lighthouse positioning system. This system uses **infrared light** to locate the drones in space. The drones have a deck that can be added to them to allow them to locate themselves in space thanks to the intersection of the light from the two base stations (Figure 8).

Components Evaluation

We have evaluated the components of the DCK to ensure that they meet the requirements of the system. The results of the evaluation through time are presented in the following sections.

We expected the drones to follow a smooth path and found that the drones were able to follow their path with a high degree of accuracy. The drones were able to maintain their position and orientation while moving along the path, demonstrating the effectiveness of the control system. (cf. Figure 9). We can note a few outliers in the path but the drones were able to maintain their position and orientation while moving along the path.



Crossing Beam Method (C.B.)

- Two LH Base Stations
- Intersection of both light planes

$$p_{1,s}, p_{2,s} = \arg \min_{p_1 \in r_{1,s}, p_2 \in r_{2,s}} \|p_1 - p_2\|_2$$

Figure 8: DCK, showing the workings of the Lighthouse positioning system. The idea is to calculate the vectors from two base stations to a sensor on the Lighthouse deck. This vector is defined by the intersection line between the two light planes of the base stations and is sometimes referred to as a “beam”, hence the name. In theory the beams should cross in the point where the sensor is located. In reality the point that is closest to both beams is used instead, and uses this as the estimated position. [19]

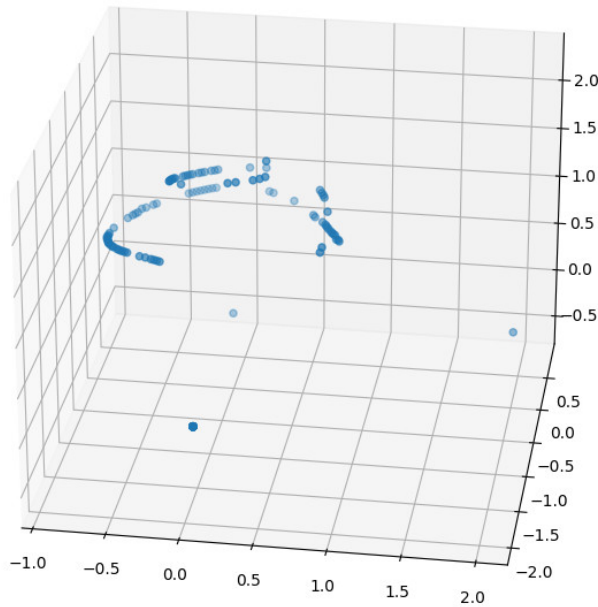


Figure 9: Drone Components evaluation. The figure shows the positions of the drones in time in 3D space coordinates. We sent the drones following a downward circular path. The drones were able to follow a path with a high degree of accuracy. We can note a few outliers in the path but the drones were able to maintain their position and orientation while moving along the path.

Architecture

The DCK is designed to provide a robust and flexible control system for managing drone swarms. The architecture of the DCK is shown in Figure 10. The main components of the system are: Web API, which provides the web-based interface for controlling the drones; Socket, which allows for lower level client drone server interface with the drones; and the DCK, which manages the communication with the drones using CRPA. The UDP server runs in parallel with the FastAPI application, thanks to *asyncio.create_task*. [20] This allows the UDP server to handle datagram messages asynchronously while the FastAPI server responds to HTTP requests.

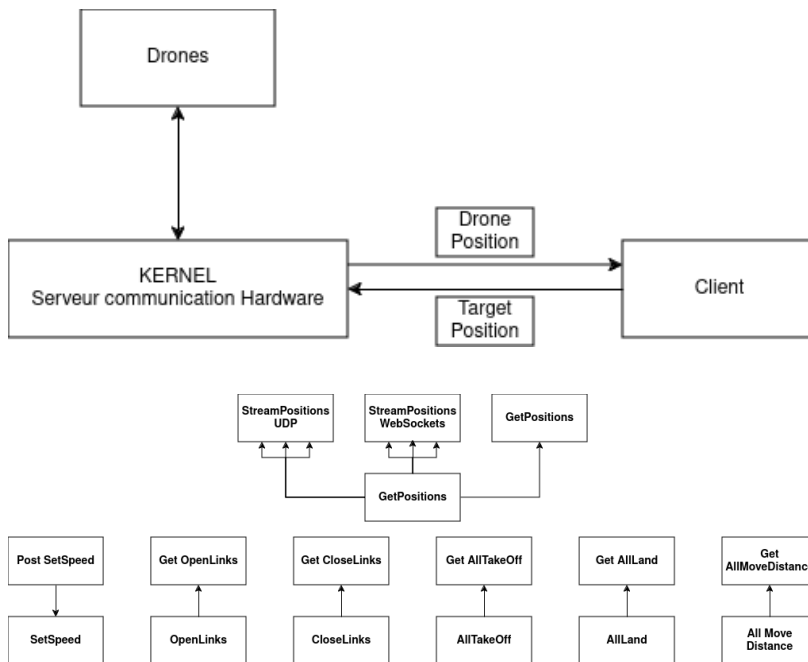


Figure 10: DCK, showing the main components of the system.

Figure 11: DCK, inside logics.

Using FastAPI we are able to provide a basic web based interface to control the drones. The UDP Client has been used to get positions of the drones without having to wait for the drones to respond or the client to send an Acknowledgment. Also it was used to lower the latency of the system and to stop the package drops errors given by the TCP server on load.

Each function in the UDP server is a coroutine, which allows for asynchronous execution. The UDP server listens for incoming messages from the drones and processes them accordingly. The FastAPI application provides the best web-based interface for controlling the drones. It exposes various endpoints for sending commands to the drones, such as takeoff, land, and move. The FastAPI application communicates with the UDP server to send commands to the drones and receive responses. The Socket component provides a lower-level interface for communicating with the drones using the Crazyflie Python library. It handles the connection to the drones and sends commands to them using the cflib API. The DCK manages the communication with the drones using the CRPA. It sends commands to the drones and receives responses from them using the Radio component (CRPA).

Also a ThreeJS interface is provided to visualize the drones in 3D space.

This interface allows users to see the drones' positions and orientations in real-time, providing a visual representation of the swarm's behavior. The ThreeJS interface communicates with the FastAPI application to receive updates on the drones' positions and orientations. (See Figure 9).

3.3 Evaluation and Results

We have realized that the DCK had a big flaw using TCP based technologies as we would observe drops in the communication. **We have then switched to UDP based technologies and have seen a big improvement in the communication.** We have also seen that the drones were able to follow a path with a high degree of accuracy. The drones were able to maintain their position and orientation while moving along the path, demonstrating the effectiveness of the control system. We were not able to break this server as we were able to send 1000 requests per second to the server and the server was able to handle them all while no function blocked the system. This was possible because the server was made as stateless as possible. So after setup no command could break it.

There is also a big improvement in the latency of getposition requests. The TCP server would take between 0.2 and 0.5 seconds to respond to a getposition request. The UDP server is able to respond in less than 0.05 seconds using cflib low level variable accessibility. This is a big improvement in the latency of the system.

3.4 Conclusion

Using the DCK, we have developed a robust and flexible control system for managing drone swarms. The system provides a **web-based interface for controlling the drones and visualizing their positions and orientations in real-time.** The system is designed to be scalable and extensible, allowing for the addition of new features and functionalities as needed. The system has been evaluated and shown to be effective in controlling drones in a swarm, with the drones able to follow a path with a high degree of accuracy using the Lighthouse positioning system. The system provides a valuable tool for developers and researchers working with drone swarms, enabling them to experiment with different control strategies and algorithms.

4 Contribution 2: Human Pose Estimation System

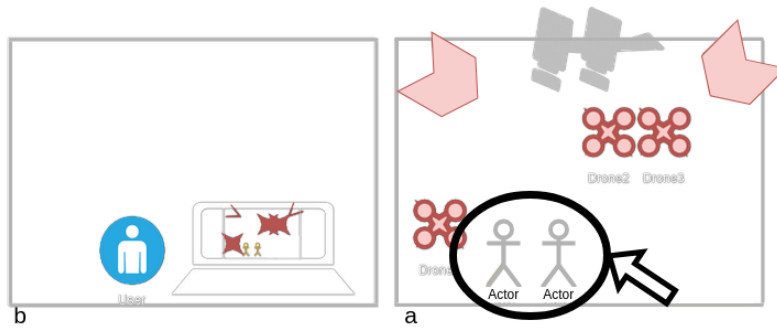


Figure 12: (a) Inside the drone room. Similarly in red is the drone system and in grey the human system with the human localization kernel. In red above there is the drone localization and control system. In grey above is the human localization system. Here we focus on the human localization kernel or system. (b) Outside the room, a user is controlling and or monitoring the drones. He is able to see the drones and humans in the 3D space using the web interface.

4.1 Introduction

In this brief section we describe the workings of the human detection system. It was developed by Etienne Pommel and adapted for our usecase. The system is based on the mediapipe library [21], which is a state-of-the-art object detection algorithm that is capable of detecting humans in real-time. [4]. The system is able to detect humans with high accuracy as they move around the environment. To the standard RGB image, the system adds depth.

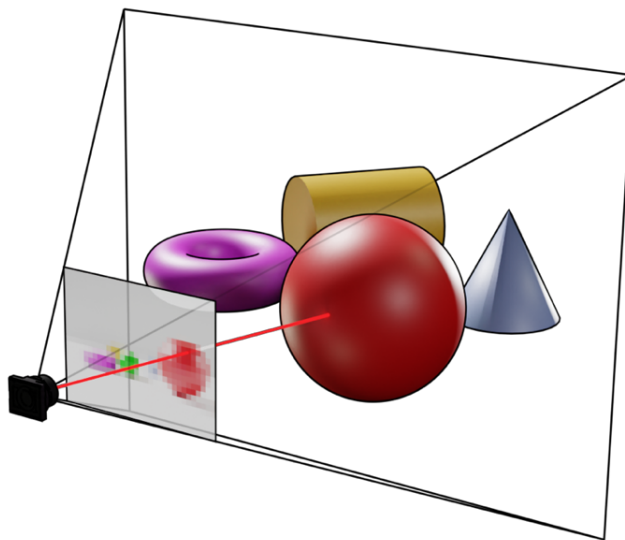


Figure 13: From Figure 4 in [16] Showing the camera in the 3D space. Note the red segment, it shows the infrared ray going to the object from the camera and estimating the distance.

The difficulties of the system are to detect the objects in the image and to estimate the distance to the object and finally to project this distance in the 3D space. Figure 16 later will show the projection of each pixel in the 3D space from Figure 13.

Here we want to detect humans in the image using of mediapipe and estimate the distance to the humans. Let us first look at the architecture of the system. The system is composed of the camera, the kernel and the client or the Web Interface in the FastAPI server see Figure 14.

The Human Localization Kernel (HLK) is responsible of the logics. The

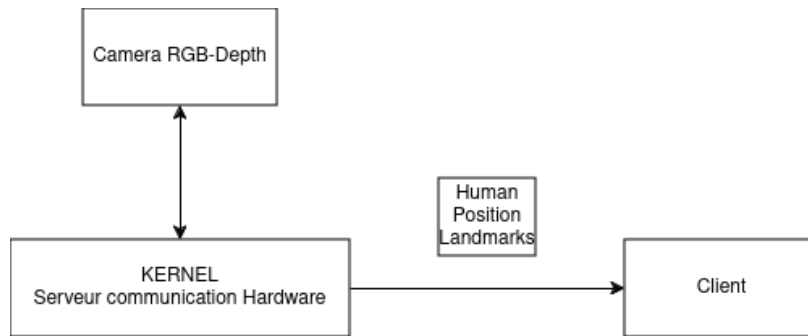


Figure 14: HLK, showing the main components of the system.

camera captures the image and depth data, which is then processed by the mediapipe library to detect humans in the image. The depth data is used to estimate the distance to the humans, which is then projected in 3D space. The FastAPI application receives the data from the mediapipe library and processes it to estimate the distance to the humans. The FastAPI application provides access to the logic and also provides a web interface to visualize the humans in 3D space.

Camera and Mediapipe Logic

The camera captures the image and depth data, which is then processed by the mediapipe library to detect humans in the image. The mediapipe library uses a state-of-the-art object detection algorithm to identify humans in the image. The depth data is used to estimate the distance to the humans, which is then projected in 3D space.

Let us look at the calibration process of the camera. The camera is calibrated using a chessboard pattern, which is placed in front of the camera. The camera captures images of the chessboard pattern from different angles, which are then used to calibrate the camera. The calibration process is repeated at regular intervals to ensure that the camera is properly calibrated.(cf Figure 15)

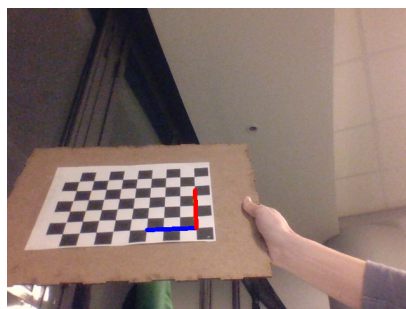


Figure 15: Chessboard with the center of the 3D space. It shows the success calibration process of the camera.

After Calibration we can project the image in the 3D space as shown in Figure 16.

Architecture of the System

The FastAPI application is responsible for providing access to our logic and providing a web interface to visualize the humans in 3D space. To allow for this we base our kernel architecture on the FastAPI application from the original algorithm from Etienne Pommel at the Institute For Future Technologies.

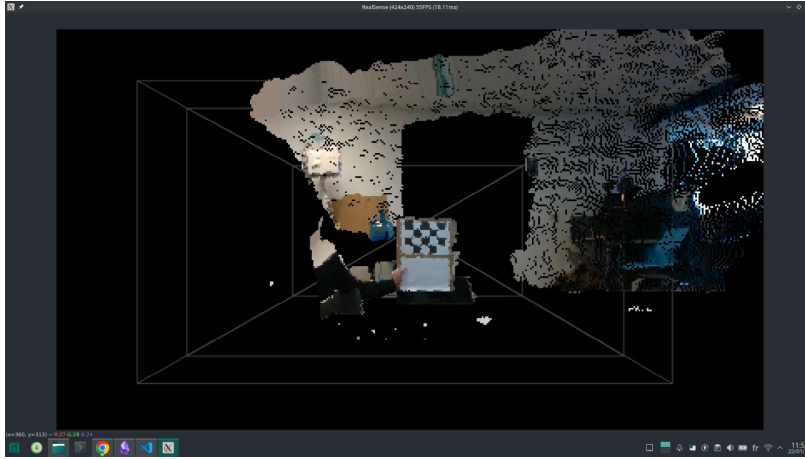


Figure 16: Intel Realsense D435, showing the workings of the depth camera and calibration capabilities, each pixel has been projected in the 3D space. The camera is able to capture the image and depth data, which is then processed. The depth data is used to estimate the distance from the pixels to humans then we are able to perform a change base to real world position from the camera RGB-D system. Which is enough data to be projected in 3D space.

The system was modified from the original algorithm as it would continuously calibrate using the chessboard. See Figure 15 and appendix 6. We modified slightly the system by moving the chessboard pose estimation thanks to a separate process. By that way the main process will not have to recompute the rotation and translation vectors for the change of basis which was the most costly part. We saved the rotation and translation vectors in global variables to be used in the main and calibration processes.

To make the system stateless, FastAPI handles two different processes. One process is to estimate the position of the humans in the 3D space. An other process allows for calibration of the camera. See Figure 17 for more.

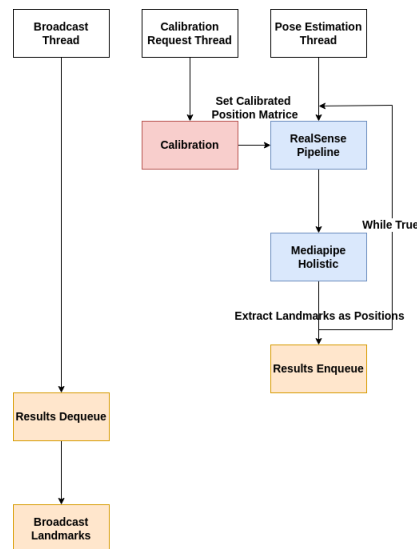


Figure 17: Object Control Kernel, inside logics. We have three separate threads/processes running. The first one sends the data to the client, the second one is the calibration process and the third one is the pose estimation process. Mediapipe runs in the pose estimation process. The calibration process is run once at the beginning at the load of the system, it can also be rerun later on when the camera moves.

Let us look at the FastAPI application. The application receives the data from the mediapipe library and processes it to estimate the distance to the humans. It also provides a ThreeJS interface to visualize the humans in 3D space. The FastAPI application is stateless and does not require any additional resources to run. It displays the humans in 3D space using the web ThreeJS interface.(cf Figure 18) We want to note that the FastAPI application is able to process the data from the mediapipe library and provide the estimated distance to the humans in real-time.

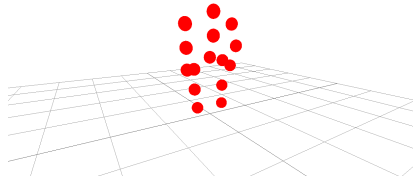


Figure 18: The web interface shows the positions of a human of the platform through its web interface run for a few seconds. It allows for the visualization of the humans in 3D space.

4.2 Evaluation and Results

A Web interface is provided to visualize the drones and humans in 3D space. This interface allows users to see the humans' positions and orientations in real-time, providing a visual representation of the system's behavior. The ThreeJS interface communicates with the kernels discussed in the paper.

We have evaluated the components of the human detection kernel to ensure that they meet the requirements of the system. The results of the evaluation are following.

We expected the measures to be as accurate as at least 5% of the real distance but expected it to be less reliable the further the subject is to the camera. In fact the system was not able to provide this much accuracy but 10% at 5 meters. The system though was able to provide a relatively good estimate of the distance to the object, constant at 12% of the real distance at 7m. The system on the other hand is precise, the estimation does not change above 1.5%. Indeed the results are less satisfying then

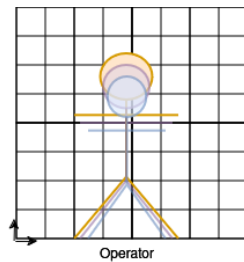


Figure 19: HLK Issue, further the object is from the camera and the center of the image, the smaller it will be estimated. The distance to the camera does not change only the size of the object in the image does. For simplicity we have used the center of the image as the center of the camera and kept the operator parallel to the camera. Distances are expected to be less accurate with an operator not parallel to the camera.

expected but the system is still able to provide a relatively good estimate of the distance to the object. The system is able to project this distance in the global reference with this 12% error which will need to be taken into account in the next part of the project.

Similarly to the drones kernel, the system was made as stateless as possible. So after setup no command could break it. This system does not require a lot of resources and is able to send all the 30 landmarks per frame at the rate of 30 per second to the client so 900 landmarks per second.

4.3 Conclusion

The human pose estimation kernel provides a valuable system for detecting and estimating the distance of humans in real-time, using the Mediapipe library in combination with depth sensors. While the results showed a degree of inaccuracy beyond 5 meters, the system successfully achieved consistent performance within 12% accuracy for shorter distances (less than 7 meters). Despite some limitations—such as reduced

reliability at greater distances and issues with object size estimation at image boundaries, the system still delivers an effective method for projecting human positions in 3D space. Future improvements should focus on enhancing the calibration mechanisms and addressing the accuracy drop at longer ranges, which will be vital for the next phase of the project.

5 Contribution 3: Platform : Human-Swarm Interaction

Let us see Figure 20 for an overview of the platform. The platform is composed of two main systems, the DCK and the HLK.

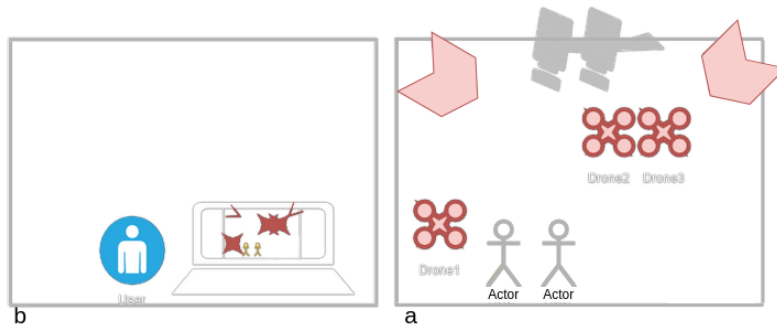


Figure 20: (a) Inside the drone room. Similarly in red is the drone system and in grey the human system with the human localization kernel. In red above there is the drone localization and control system. In grey above is the human localization system. (b) Outside the room, a user is controlling and/or monitoring the drones. He is able to see the drones and humans in the 3D space using the web interface.

5.1 Introduction

In this section, we present our third contribution, which is the platform for human-swarm interaction. We present the setup and the software architecture that we have developed to control drones indoors around humans. We also present the results of the evaluation of the platform.

Our solution works on premise with a web-based interface to control the drones as it requires real-time computation. **Both for drone control and human pose detection computations have to be synchronized** as both subjects are in the same environment. The two systems are stateless allowing for a more flexible and scalable system. **They have to run on premise to keep them synchronized.** The drones are controlled using the DCK and the human operator is localized using the HLK. The two systems work together to ensure that the drones can navigate the environment safely and efficiently while interacting with the human operator. The main challenge is to have **the two systems work together in real-time** to ensure that the drones can avoid collisions with the human operator and other obstacles in the environment.

5.2 Platform for Human-Swarm Interaction

Our system is especially useful for testing new control strategies and algorithms in a safe and controlled environment. The platform provides a web-based interface for controlling the drones and visualizing their positions and orientations in real-time. The platform also provides a 3D visualization of the drones in the environment, allowing users to see the drones' positions and orientations in real-time. The platform is designed to be flexible and extensible, allowing for the addition of new features and functionalities as needed. The web interface acts as a monitoring tool for the platform. See Figure 21 for the positions of the elements of the platform through its web interface run for a few seconds.

The platform is composed of two main systems: the drone control kernel (DCK) and the human localization kernel (HLK). The DCK is responsible for controlling the drones and ensuring that they move safely and efficiently in the environment. The HLK is responsible for detecting

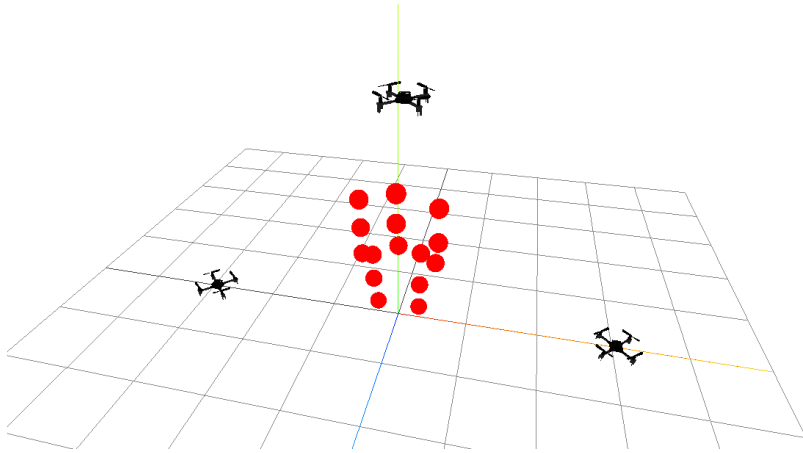


Figure 21: Shows the positions of the elements of the platform through its web interface run for a few seconds.

the human operator's position and orientation in the environment and ensuring that the drones interact with the human operator in a safe and controlled manner. The two systems work together to ensure that the drones can navigate the environment safely and efficiently while interacting with the human operator. The main challenge is to have the two systems work together in real-time to ensure that the drones can avoid collisions with the human operator and other obstacles in the environment.

A user would then want to set speeds through his own app.

5.3 Opportunitites

We have successfully built a usable platform with known people and subjects. We plan on testing it further and giving it to students to test further and develop on. Students can access the platform and develop their own algorithms to control the drones and have them interact with them in real-time. This will allow students to experiment with different control strategies and learn about the challenges of controlling drones in indoor environments especially without having to work on the hardware part of the drones.

One student Maximilien Menesguen [22] developed the famous boids [23] algorithm in Unity to make the drones follow a moving point in space using our UDP server, a work in progress. One of the main challenges in controlling drones indoors around humans is ensuring that the drones can avoid collisions with people and objects. The system continuously monitors the surroundings and adjusts the simulated drones' trajectories to avoid collisions. it is based on the original boids algorithm [23]. The boids algorithm is a simple and elegant model for simulating the flocking behavior of birds. It is based on three simple rules: separation, alignment, and cohesion. The separation rule ensures that the drones maintain a safe distance from each other, the alignment rule ensures that the drones move in the same direction, and the cohesion rule ensures that the drones stay together as a group. By following these rules, the drones can navigate the environment safely and efficiently.

The alignment rule is to be modified to take into account the position of the human operator so the drones interact more with the human. Thus we plan on setting the red Ball of the UAVs to be the user. This modification

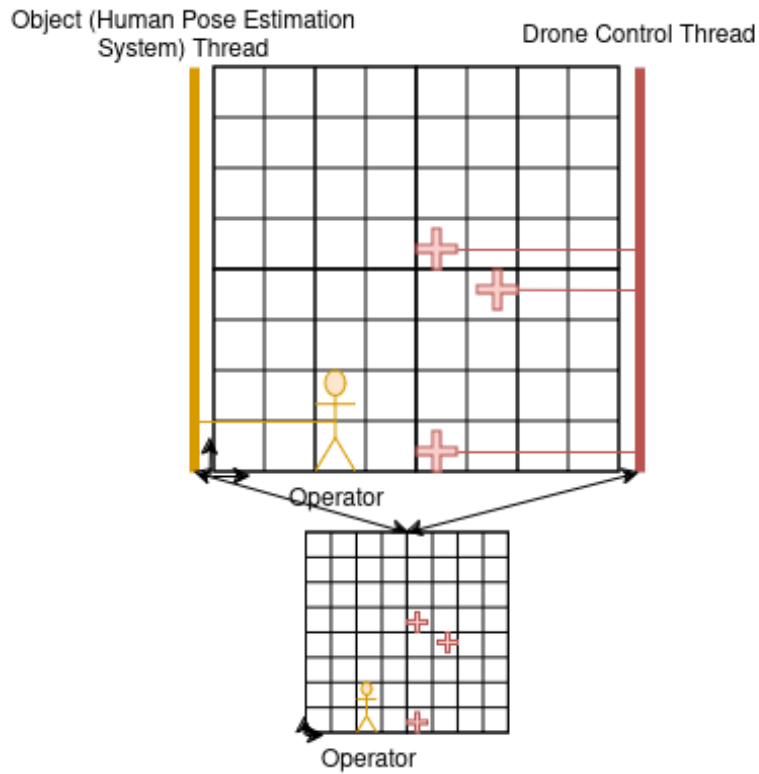


Figure 22: Shows the functionalities of the platform through its web interface and double threads. Above, the realworld view and below the estimation displayed on the operator's screen. DCK and HLK run in parallel

allows the drones to follow the human operator while maintaining a safe distance from them. The cohesion rule has also been modified to ensure that the drones stay within a predefined area around the human operators. This last modification would allow the drones to move around the human operator without staying too far from them.

We plan to add the repulsion rule directly to the human operator and the drones. This rule would ensure that the drones maintain a safe distance from the human operator at all times. Adding the security layer to the platform is also a priority. To see the algorithm in simulation, see Figure 23.

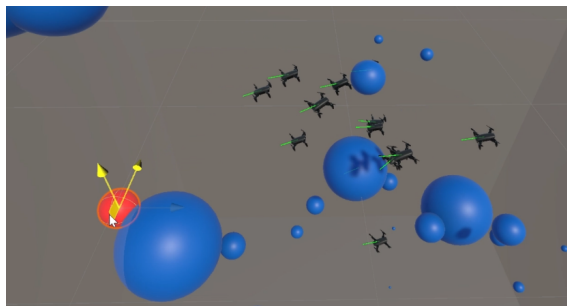


Figure 23: Drones following a moving point in space. In blue the obstacles and in red the human operator to follow.

6 Conclusion and Opportunities for Future Work

This technology gives possibilities in developpement to warehouses for adding drone carriers around sensible objects or animals.

Limits and Future Work

As discussed previously in the paper, the current platform has some limitations that need to be addressed in future work. One of the main limitations is the reliance on lighthouse technology [24] for positioning. While it uses sensors that are effective, they are light sensitive and may not be suitable for all indoor environments especially close to windows. Future work will focus on developing around a more robust location system that can adapt to different environments and scenarios.

There is also a need to provide a cybersecurity layer to the platform to ensure that the drones are secure from external threats. This will involve developing a secure communication protocol to protect the drones from unauthorized access.

Finally the platform needs to be tested in a real-world environment to evaluate its performance and reliability. This will involve conducting experiments in different indoor environments to assess the platform's intuitiveness for students. The results of these experiments will help to identify any issues with the platform and guide future development efforts.

Appendix

Pose estimation pipeline

This system developed at the Institute for Future Technologies by Etienne Pommel detects human body landmarks using mediapipe[21] with 2D Camera Images. The system then projects these landmarks into 3D space using depth data from a RealSense camera.

See in Figure 24 the complete pipeline of the system. It is composed of three main elements: Chessboard Pose estimation, Human body pose estimation using mediapipe and finally the change of basis. The camera actually is calibrated once to get its distortion coefficients and camera matrix. We run though find chessboard corners and then use the solvePnP function to get the pose of the chessboard. We then use the Rodrigues function to get the rotation matrix and the translation vector. After all these processings we use the change of basis to get the human pose in the same basis as the chessboard. See Figure 24 for the complete pipeline.

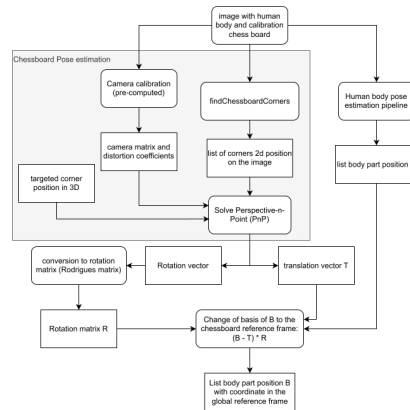


Figure 24: Human Localization and pose estimation inside logics. Notice here the main elements, Chessboard Pose estimation, Human body pose estimation using mediapipe and finally the change of basis.

The system leverages pose estimation to detect human body landmarks using computer vision, specifically mediapipe [21]. First, a camera captures RGB and depth data from the environment, with the depth data being crucial for calculating the distance of each detected point. The system uses a monocular camera setup, and it processes images through MediaPipe's holistic model, which detects human body landmarks, including key body joints like the head, shoulders, and limbs. These 2D landmarks are then projected into 3D space using depth data from the RealSense camera.

The system performs camera calibration to compute intrinsic parameters such as the camera matrix and distortion coefficients. Calibration ensures that the system accurately maps 2D pixel coordinates to real-world 3D points. OpenCV's solvePnP function computes the human's pose (rotation and translation vectors) relative to the camera using chessboard patterns, which serve as a known reference in the scene. After obtaining the pose, the Rodrigues rotation matrix is used to transform the body landmarks into the coordinate system of the chessboard, essentially re-aligning the 3D positions of the detected human body with the physical world.

This process ensures that the system can track human movement in 3D space accurately, which is crucial for human-drone interaction and control.

Acknowledgment

A Testbed Environment for Task Development was developed at the IFT. Marc Teyssier, Xiao Xiao and Clement Duhart showed much faith to enable such a project.

Etienne Pommel and Maximilien Menesguen of the Institute for future technologies, thank you for joining yourselves to me. Your enthusiasm and dedication to the project have been a great source of inspiration.

Thomas Carstens and Nicolas Stas of the Institute for future technologies have been a great help in the development of the platform. Your technical expertise and guidance have been invaluable.

References

Here are the references in citation order.

- [1] Manuele Brambilla et al. 'Swarm robotics: A review from the swarm engineering perspective'. In: *Swarm Intelligence* 7.1 (2013), pp. 1–41 (cited on pages 1, 2).
- [2] J. A. Preiss et al. 'CrazySwarm: A large nano-quadcopter swarm'. In: *2017 IEEE International Conference on Robotics and Automation (ICRA)*. 2017 (cited on pages 2, 3, 5, 7).
- [3] Bitcraze. CFLib - Python library for Crazyflie. <https://www.bitcraze.io/documentation/repository/crazyflie-lib-python/master/>. Accessed: 2024-10-10. 2020 (cited on pages 2, 3, 6, 7).
- [4] Mediapipe Dev Team. 'Real-time pose estimation using Intel RealSense and MediaPipe for human-drone interaction'. In: (2020) (cited on pages 3, 6, 7, 13).
- [5] Chien Nguyen, Pascal Fua, et al. 'FlyJacket: An upper body soft exoskeleton for immersive drone control'. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2018 (cited on pages 3, 5–7).
- [6] Maziar Shahbazi et al. 'A review of human-drone interaction: Methods and applications'. In: *Computers & Human Behavior* 106 (2020), p. 106270 (cited on page 3).
- [7] Jessica R. Cauchard and et al. 'Drone & Me: An Exploration Into Natural Human-Drone Interaction'. In: *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* (2015) (cited on pages 4, 5).
- [8] X. Ju et al. 'Safe human-drone interaction in indoor environments'. In: *International Conference on Human-Robot Interaction (HRI)*. 2017 (cited on page 5).
- [9] L. Cai and et al. 'Human-drone interaction based on posture recognition with monocular vision'. In: *Journal of Intelligent & Robotic Systems* (2019) (cited on pages 5, 6).
- [10] M. Rubenstein et al. 'Kilobot: A low-cost scalable robot system for collective behaviors'. In: *2012 IEEE International Conference on Robotics and Automation (ICRA)*. 2012 (cited on page 5).
- [11] David Saldana et al. 'ModQuad: The Flying Modular Structure that Self-Assembles in Midair'. en. In: *2018 IEEE International Conference on Robotics and Automation (ICRA)*. Brisbane, QLD: IEEE, May 2018, pp. 691–698. doi: [10.1109/ICRA.2018.8461014](https://doi.org/10.1109/ICRA.2018.8461014). (Visited on 10/10/2024) (cited on pages 5, 7).
- [12] J. L. Sanchez-Lopez et al. 'CrazyChoir: Artistic performance with multiple Crazyflie drones'. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2022 (cited on page 5).
- [13] James A. Preiss et al. 'Crazyswarm: A large nano-quadcopter swarm'. In: *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE. 2017, pp. 3299–3304 (cited on page 5).
- [14] Thomas Carstens. 'Intelligent Systems for Next Generation Drone Functionality'. en. In: () (cited on pages 6, 9).
- [15] *Feels like flying: FlyJacket exosuit lets you control a drone with your body*. en-US. Section: Drones. Apr. 2018. URL: <https://newatlas.com/flyjacket-drone-exosuit/54260/> (visited on 10/28/2024) (cited on page 6).
- [16] *Projection, Texture-Mapping and Occlusion with Intel® RealSense™ Depth Cameras*. en. URL: <https://dev.intelrealsense.com/docs/projection-texture-mapping-and-occlusion-with-intel-realsense-depth-cameras> (visited on 10/27/2024) (cited on pages 6, 13).
- [17] Amine Tayebi. 'Gesture-based drone control with Leap Motion and Arduino'. In: *International Journal of Advanced Research in Robotics* (2020) (cited on page 6).
- [18] *Overview — Crazyswarm 0.3 documentation*. URL: <https://crazyswarm.readthedocs.io/en/latest/overview.html> (visited on 09/14/2023) (cited on page 7).
- [19] *Lighthouse positioning methods | Bitcraze*. URL: https://www.bitcraze.io/documentation/repository/crazyflie-firmware/master/functional-areas/lighthouse/positioning_methods/ (visited on 10/15/2024) (cited on page 10).
- [20] Python Software Foundation. 'asyncio: Asynchronous I/O, event loop, coroutines and tasks'. In: (2022). Accessed: 2024-10-10 (cited on page 11).
- [21] Google Research. *MediaPipe: A Framework for Building Perception Pipelines*. <https://google.github.io/mediapipe/>. Accessed: 2024-10-10. 2020 (cited on pages 13, 22).

- [22] Maximilien Menesguen. Maximilien Menesguen. <https://ift.devinci.fr/member/maximilien-menesguen>. Accessed: 2024-10-10. 2023 (cited on page 19).
- [23] Craig W. Reynolds. 'Flocks, Herds, and Schools: A Distributed Behavioral Model'. In: Computer Graphics 21.4 (1987), pp. 25–34 (cited on page 19).
- [24] DVIC - De Vinci Innovation Center. URL: <https://dvic.devinci.fr/tutorial/lighthouse-positioning> (visited on 09/07/2023) (cited on page 21).

Special Terms

Numbers

3D Three Dimensional.. 2, 8, 11, 13–18

A

API Application Programming Interface (a set of tools for building software).. 8, 9, 11

C

cflib Crazyflie Library (Python-based library for drone control).. 8, 11, 12

CRPA Crazy Radio PA (a radio dongle used to communicate with Crazyflie drones).. 9, 11

D

DCK Drone Control Kernel (a module for swarm drones control).. 1, 8–12, 18

F

FastAPI A modern, fast (high-performance), web framework for building APIs with Python 3.6+ based on standard Python type hints.. 11–15

H

HLK Human Localization Kernel (a module for human localization and pose estimation).. 14, 16, 18

L

Lighthouse A base station used for tracking the position of VR devices.. 9, 10, 12

R

RGB Red, Green, Blue (standard color model for digital images).. 6, 13

RGB-D Red, Green, Blue with Depth (imaging that captures depth along with standard color).. 6

ROS Robot Operating System (a flexible framework for writing robot software).. 5, 6

S

stateless A system that does not retain information between requests.. 15

T

ThreeJS A JavaScript library used to create and display animated 3D computer graphics in a web browser.. 11, 12, 15, 16